cxxc7 Merge branch 'main' of https://github.com/cxxc7/yantra-chatbot 072029d · last month

1238 lines (1110 loc) · 42 KB

Code Blame Raw

```
1   "use client";
2
3   declare global {
4     interface Window {
5       SpeechRecognition?: { new (): SpeechRecognition } | undefined;
6       webkitSpeechRecognition?: { new (): SpeechRecognition } | undefined;
7     }
8
9     interface SpeechRecognitionEvent {
10      readonly resultIndex: number;
11      readonly results: SpeechRecognitionResultList;
12    }
13
14    interface SpeechRecognitionResult {
15      readonly isFinal: boolean;
16      readonly 0: { transcript: string };
17      readonly length: number;
18      [index: number]: { transcript: string; confidence?: number };
19    }
20
21    interface SpeechRecognitionResultList {
22      readonly length: number;
23      item(index: number): SpeechRecognitionResult;
24      [index: number]: SpeechRecognitionResult;
25    }
26
27    interface SpeechRecognition {
28      lang: string;
29      interimResults: boolean;
30      maxAlternatives: number;
31      onresult: ((ev: SpeechRecognitionEvent) => any) | null;
32      onend: (() => any) | null;
33      onerror: ((ev: any) => any) | null;
34      start(): void;
35      stop(): void;
36      abort?(): void;
37    }
38  }
39
40  import React, {
41    useEffect,
42    useMemo,
43    useRef,
44    useState,
45  } from "react";
```

```tsx
import ReactMarkdown from "react-markdown";
import remarkGfm from "remark-gfm";

type Role = "user" | "assistant";

interface Message {
  id: string;
  role: Role;
  content: string;
  timestamp: string;
}

interface ChatApiResponse {
  answer: string;
  used_context: string[];
  from_fallback: boolean;
  brochure_url?: string | null;
}

type Theme = "dark" | "light";

const API_BASE_URL =
  process.env.NEXT_PUBLIC_API_BASE_URL || "http://127.0.0.1:8000";

const SUGGESTED_QUESTIONS: string[] = [
  "What is the chisel diameter for Hyundai R30?",
  "Which breaker is compatible with SANY SY20?",
  "Compare JCB 3DX and Hyundai R30.",
  "Which is the best breaker option for Hyundai R30?",
];

const FOLLOW_UP_LINES: string[] = [
  "Anything else I can help you with?",
  "Want to compare any other models?",
  "Need details on spare parts or dealers next?",
];

function getGreeting(): string {
  const hour = new Date().getHours();
  if (hour < 12) return "Good morning";
  if (hour < 17) return "Good afternoon";
  return "Good evening";
}

function randomFrom<T>(arr: T[]): T {
  return arr[Math.floor(Math.random() * arr.length)];
}

function formatTime(date: Date = new Date()): string {
  return date.toLocaleTimeString([], {
    hour: "2-digit",
    minute: "2-digit",
  });
}

function looksLikePipeTable(text: string): boolean {
  const lines = text.split("\n").map((l) => l.trim());
  const pipeLines = lines.filter((l) => l.includes("|"));
  if (pipeLines.length < 2) return false;
  for (const l of lines) {
    if (/^(\|?\s*:?-+:?\s*\|)+\s*:?-+:?\s*\|?\s*$/.test(l)) return true;
    if (/^-{3,}\s*\|/.test(l) || /\|\s*-{3,}/.test(l)) return true;
  }
```

```
    return false;
  }

  function boldParameterNames(content: string): string {
    if (!content) return content;
    const lines = content.split(/\n/).map((line) => {
      if (line.includes("|")) return line;
      if (/^\s*([-*]|\d+\.)\s/.test(line)) return line;
      return line.replace(
        /^([A-Za-z0-9 _/()&%-]+):\s*(.+)$/g,
        (_m, key, rest) => `**${key.trim()}**: ${rest.trim()}`
      );
    });
    return lines.join("\n");
  }

  function preprocessAssistant(content: string): string {
    if (!content) return content;
    let out = content;

    const fencedMatch = out.match(/^\s*```(?:\w+)?\n([\s\S]*?)\n```s*$/);
    if (fencedMatch) {
      out = fencedMatch[1];
    }

    const headerPatterns: RegExp[] = [
      /(Compatible machines:)(?!\n)/i,
      /(Here are the details[^\n]*?breaker model:)(?!\n)/i,
      /(Here are the details[^\n]*?breaker:)(?!\n)/i,
    ];
    headerPatterns.forEach((pat) => {
      out = out.replace(pat, (_m, g1) => `${g1}\n\n`);
    });

    out = out.replace(
      /(:\n[^\n]*?)(\b[A-Za-z][A-Za-z0-9 _/()&%-]{0,40}):\s)/g,
      (m, before, secondKey) => {
        return `${before}\n${secondKey}`;
      }
    );

    out = out.replace(/(Compatible machines:\n?)([^\n]+)/i, (m, head, list) => {
      const items = list
        .split(/[,;]+/)
        .map((s: string) => s.trim())
        .filter((v: string) => Boolean(v));
      if (items.length === 0) return m;
      const heading = "**COMPATIBLE MACHINES:**";
      const lines = items.map((i: string) => `- ${i}`);
      return `${heading}\n\n${lines.join("\n")}`;
    });

    out = out.replace(/(^|\n)Compatible machines:\s*\n+/i, (m) => {
      return `${m.startsWith("\n") ? "\n" : ""}**COMPATIBLE MACHINES:**\n\n`;
    });

    const lines = out.split(/\n/);
    let inCompat = false;
    for (let idx = 0; idx < lines.length; idx++) {
      const line = lines[idx];
      if (/^\*\*COMPATIBLE MACHINES:\*\*$/.test(line.trim())) {
        inCompat = true;
        continue;
```

```
        }
        if (inCompat) {
          if (!line.trim()) continue;
          if (/^\*\*.+\*\*$/.test(line.trim()) || /^[A-Za-z0-9 _/()&%-]+:\s/.test(line))
            if (!/^\s*[-]/.test(line.trim()) && !/^Machine:\s/.test(line.trim())) inCompa
            continue;
          }
          if (/^[-*]\s+/.test(line)) {
            const name = line.replace(/^[-*]\s+/, "").trim();
            lines[idx] = `- ${name}`;
            continue;
          }
          if (!/^-/.test(line) && /^[A-Za-z0-9 .()/-]{2,}$/.test(line.trim())) {
            lines[idx] = `- ${line.trim()}`;
            continue;
          }
          inCompat = false;
        }
      }
      out = lines.join("\n");

      const allLines = out.split("\n");
      let i = 0;
      const newLines: string[] = [];
      while (i < allLines.length) {
        if (allLines[i].includes("|")) {
          const start = i;
          let end = i;
          while (end + 1 < allLines.length && allLines[end + 1].includes("|")) end++;
          const block = allLines.slice(start, end + 1).join("\n");
          if (looksLikePipeTable(block)) {
            if (newLines.length > 0 && newLines[newLines.length - 1].trim() !== "") {
              newLines.push("");
            }
            const cleanedBlock = block
              .replace(/^\s*```/g, "")
              .replace(/```\s*$/g, "");
            newLines.push(cleanedBlock);
            if (end + 1 < allLines.length && allLines[end + 1].trim() !== "") {
              newLines.push("");
            }
            i = end + 1;
            continue;
          } else {
            newLines.push(allLines[i]);
            i++;
            continue;
          }
        } else {
          newLines.push(allLines[i]);
          i++;
        }
      }

      out = newLines.join("\n");

      return out;
    }

    export default function YantraChatPage() {
      const [messages, setMessages] = useState<Message[]>([]);
      const [input, setInput] = useState("");
      const [loading, setLoading] = useState(false);
```

```
235       const [greeting, setGreeting] = useState("Hello");
236       const [theme, setTheme] = useState<Theme>("dark");
237       const textareaRef = useRef<HTMLTextAreaElement | null>(null);
238
239       // microphone / speech recognition state
240       const [isRecording, setIsRecording] = useState(false);
241       const recognitionRef = useRef<SpeechRecognition | null>(null);
242       // base value of textarea when recording started
243       const baseBeforeRecordingRef = useRef<string>("");
244       // current interim visible text (not committed to textarea until final)
245       const [interimText, setInterimText] = useState<string>("");
246       // flag when permission denied
247       const [micPermissionDenied, setMicPermissionDenied] = useState<boolean>(false);
248
249       // server recording state (MediaRecorder)
250       const mediaRecorderRef = useRef<MediaRecorder | null>(null);
251       const audioChunksRef = useRef<Blob[]>([]);
252       const [isServerRecording, setIsServerRecording] = useState(false);
253       const [isTranscribing, setIsTranscribing] = useState(false);
254
255       // client-only mount + support detection
256       const [isMounted, setIsMounted] = useState(false);
257       const [supportsSpeechRecognition, setSupportsSpeechRecognition] = useState(false);
258
259       // language selector: 'en' | 'hi' | 'kn'
260       const [language, setLanguage] = useState<"en" | "hi" | "kn">("en");
261       // force server STT override
262       const [forceServerStt, setForceServerStt] = useState<boolean>(false);
263
264       // brochure modal state
265       const [brochureOpen, setBrochureOpen] = useState(false);
266       const [brochureUrl, setBrochureUrl] = useState<string | null>(null);
267
268       // refs to control scrolling
269       const messagesContainerRef = useRef<HTMLDivElement | null>(null);
270       const messagesEndRef = useRef<HTMLDivElement | null>(null);
271
272       useEffect(() => {
273         setGreeting(getGreeting());
274       }, []);
275
276       useEffect(() => {
277         if (typeof window === "undefined") return;
278         const stored = window.localStorage.getItem("yantra-theme");
279         if (stored === "dark" || stored === "light") {
280           setTheme(stored);
281         }
282       }, []);
283
284       useEffect(() => {
285         if (typeof window === "undefined") return;
286         try {
287           const raw = window.localStorage.getItem("yantra_chat_messages_v1");
288           if (raw) {
289             const parsed = JSON.parse(raw) as Message[];
290             if (Array.isArray(parsed)) {
291               setMessages(parsed);
292             }
293           }
294         } catch (e) {
295           console.warn("Failed to restore messages", e);
296         }
297       }, []);
```

```
298
299       useEffect(() => {
300         if (typeof window === "undefined") return;
301         try {
302           window.localStorage.setItem("yantra_chat_messages_v1", JSON.stringify(messages)
303         } catch (e) {
304           console.warn("Failed to save messages", e);
305         }
306       }, [messages]);
307
308       useEffect(() => {
309         if (messagesContainerRef.current) {
310           setTimeout(() => {
311             try {
312               messagesContainerRef.current!.scrollTo({
313                 top: messagesContainerRef.current!.scrollHeight,
314                 behavior: "smooth",
315               });
316             } catch {
317               messagesEndRef.current?.scrollIntoView({ behavior: "smooth" });
318             }
319           }, 50);
320         } else {
321           messagesEndRef.current?.scrollIntoView({ behavior: "smooth" });
322         }
323       }, [messages]);
324
325       const quickActions = useMemo(() => SUGGESTED_QUESTIONS, []);
326
327       const toggleTheme = () => {
328         setTheme((prev) => {
329           const next: Theme = prev === "dark" ? "light" : "dark";
330           if (typeof window !== "undefined") {
331             window.localStorage.setItem("yantra-theme", next);
332           }
333           return next;
334         });
335       };
336
337       const handleClearChat = () => {
338         setMessages([]);
339         setInput("");
340         textareaRef.current?.focus();
341         try {
342           if (typeof window !== "undefined") {
343             window.localStorage.removeItem("yantra_chat_messages_v1");
344           }
345         } catch {}
346       };
347
348       const handleCopy = (content: string) => {
349         if (typeof navigator === "undefined" || !navigator.clipboard) return;
350         navigator.clipboard.writeText(content).catch((err) => {
351           console.error("Failed to copy", err);
352         });
353       };
354
355       // open brochure modal (normalize href first)
356       const openBrochureModal = (href: string) => {
357         try {
358           const u = new URL(href, window.location.href);
359           let final = u.href;
360           if (u.pathname.startsWith("/brochures/view/") || u.pathname.startsWith("/brochu
```

```
            if (!final.includes("#")) final = `${final}#toolbar=1`;
          }
          setBrochureUrl(final);
          setBrochureOpen(true);
        } catch (e) {
          setBrochureUrl(href);
          setBrochureOpen(true);
        }
      };

      const closeBrochureModal = () => {
        setBrochureOpen(false);
        setTimeout(() => setBrochureUrl(null), 200);
      };

      const sendMessage = async (text: string) => {
        const trimmed = text.trim();
        if (!trimmed) return;

        const now = formatTime();
        const lc = trimmed.toLowerCase();

        if (lc.includes("thank you") || lc.includes("thanks")) {
          const userMsg: Message = {
            id: crypto.randomUUID(),
            role: "user",
            content: trimmed,
            timestamp: now,
          };
          const botMsg: Message = {
            id: crypto.randomUUID(),
            role: "assistant",
            content:
              "You're welcome! Anything else I can help you with today?",
            timestamp: formatTime(),
          };
          setMessages((prev) => [...prev, userMsg, botMsg]);
          setInput("");
          textareaRef.current?.focus();
          return;
        }

        const userMsg: Message = {
          id: crypto.randomUUID(),
          role: "user",
          content: trimmed,
          timestamp: now,
        };

        const newMessages = [...messages, userMsg];
        setMessages(newMessages);
        setInput("");
        setLoading(true);

        try {
          const backendMessages = newMessages.map((m) => ({
            role: m.role,
            content: m.content,
          }));

          const res = await fetch(`${API_BASE_URL}/api/chat`, {
            method: "POST",
            headers: {
```

```
424               "Content-Type": "application/json",
425             },
426             body: JSON.stringify({ messages: backendMessages }),
427           });
428
429           if (!res.ok) {
430             const data = await res.json().catch(() => ({}));
431             throw new Error(data.detail || "Chat request failed");
432           }
433
434           const data: ChatApiResponse = await res.json();
435
436           let combined = data.answer.trim();
437           const followup = randomFrom(FOLLOW_UP_LINES);
438           combined = `${combined}\n\n_${followup}_`;
439
440           // Append the brochure link in the message, but DO NOT auto-open the modal.
441           if (data.brochure_url) {
442             combined = `${combined}\n\n[📘 Open Brochure](${data.brochure_url})`;
443           }
444
445           const botMsg: Message = {
446             id: crypto.randomUUID(),
447             role: "assistant",
448             content: combined,
449             timestamp: formatTime(),
450           };
451
452           setMessages((prev) => [...prev, botMsg]);
453
454         } catch (err: any) {
455           console.error(err);
456           const errorMsg: Message = {
457             id: crypto.randomUUID(),
458             role: "assistant",
459             content: `Error: ${err.message || "Something went wrong"}`,
460             timestamp: formatTime(),
461           };
462           setMessages((prev) => [...prev, errorMsg]);
463         } finally {
464           setLoading(false);
465           textareaRef.current?.focus();
466         }
467       };
468
469       const handleSend = async () => {
470         if (loading) return;
471         await sendMessage(input);
472       };
473
474       const handleQuickQuestion = (q: string) => {
475         if (loading) return;
476         setInput("");
477         void sendMessage(q);
478       };
479
480       const handleKeyDown = (e: React.KeyboardEvent<HTMLTextAreaElement>) => {
481         if (e.key === "Enter" && !e.shiftKey) {
482           e.preventDefault();
483           if (!loading) {
484             void handleSend();
485           }
486         }
```

```
    };

    const hasMessages = messages.length > 0;

    const rootClass =
      theme === "dark"
        ? "min-h-screen flex flex-col bg-slate-950 text-slate-100"
        : "min-h-screen flex flex-col bg-slate-50 text-slate-900";

    const headerBorder =
      theme === "dark" ? "border-slate-800" : "border-slate-200";

    const headerBg =
      theme === "dark"
        ? "bg-gradient-to-r from-slate-950 via-slate-900 to-slate-950"
        : "bg-white";

    const headerSubText =
      theme === "dark" ? "text-slate-400" : "text-slate-500";

    const cardClass =
      theme === "dark"
        ? "rounded-2xl border border-slate-800/80 bg-slate-900/60 shadow-[0_20px_60px_r
        : "rounded-2xl border border-slate-200 bg-white shadow-[0_20px_40px_rgba(15,23,

    const dividerBorder =
      theme === "dark" ? "border-slate-800/80" : "border-slate-200";

    const assistantBubbleClass =
      theme === "dark"
        ? "bg-slate-800/90 text-slate-100 border border-slate-700/60"
        : "bg-slate-100 text-slate-900 border border-slate-200";

    const timestampClass =
      theme === "dark" ? "text-slate-500" : "text-slate-400";

    const inputBorder =
      theme === "dark" ? "border-slate-800" : "border-slate-300";

    const inputBg =
      theme === "dark" ? "bg-slate-900/80" : "bg-white";

    // ------- Speech recognition helpers -------
    useEffect(() => {
      // mark mounted and run feature detection on client only
      setIsMounted(true);
      const supported = typeof window !== "undefined" && (!!(window as any).SpeechRecog
      setSupportsSpeechRecognition(Boolean(supported));
    }, []);

    useEffect(() => {
      if (!supportsSpeechRecognition) return;

      const SpeechRecognitionConstructor =
        (window as any).SpeechRecognition || (window as any).webkitSpeechRecognition;

      const rec: SpeechRecognition = new SpeechRecognitionConstructor();
      rec.lang = "en-IN";
      rec.interimResults = true;
      rec.maxAlternatives = 1;

      recognitionRef.current = rec;
```

```
      rec.onresult = (ev: SpeechRecognitionEvent) => {
        // Build interim + final from results. We DO NOT mutate the textarea's value fo
        let interim = "";
        let final = "";
        for (let i = 0; i < ev.results.length; i++) {
          const r = ev.results[i];
          if (r.isFinal) {
            final += r[0].transcript;
          } else {
            interim += r[0].transcript;
          }
        }

        setInterimText(interim || "");

        // If final result present, commit to textarea `input` (append to whatever is c
        if (final && final.trim()) {
          setInput((prev) => {
            const appended = (prev + (prev && !prev.endsWith(" ") ? " " : "") + final).
            return appended + " ";
          });
          baseBeforeRecordingRef.current = "";
          setInterimText("");
        }
      };

      rec.onend = () => {
        setIsRecording(false);
        setInterimText("");
        baseBeforeRecordingRef.current = "";
      };

      rec.onerror = (err: any) => {
        console.error("SpeechRecognition error", err);
        try {
          const name = err?.error || err?.name || "";
          if (typeof name === "string" && (name.toLowerCase().includes("not-allowed") |
            setMicPermissionDenied(true);
          }
        } catch {}
        setIsRecording(false);
        setInterimText("");
        baseBeforeRecordingRef.current = "";
      };

      return () => {
        try {
          rec.onresult = null;
          rec.onend = null;
          rec.onerror = null;
          recognitionRef.current = null;
        } catch {}
      };
    // only re-run when supportsSpeechRecognition flips
    }, [supportsSpeechRecognition]);

    // Decide recording strategy: automatic
    const shouldUseServerStt = (lang: string, forceServer: boolean) => {
      // Auto rules:
      // - If forceServer true => server
      // - If Kannada (kn) => server (browser STT rarely supports)
      // - Else if browser supports SpeechRecognition => use it
      // - else => server
```

```
613         if (forceServer) return true;
614         if (lang === "kn") return true;
615         if (supportsSpeechRecognition) return false;
616         return true;
617       };
618
619       // -------- Server recording (MediaRecorder) flow --------
620       const startServerRecording = async () => {
621         try {
622           setMicPermissionDenied(false);
623           audioChunksRef.current = [];
624           const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
625           const mr = new MediaRecorder(stream, { mimeType: "audio/webm" });
626           mediaRecorderRef.current = mr;
627           setIsServerRecording(true);
628           setIsTranscribing(false);
629           audioChunksRef.current = [];
630
631           mr.ondataavailable = (ev: BlobEvent) => {
632             if (ev.data && ev.data.size > 0) {
633               audioChunksRef.current.push(ev.data);
634             }
635           };
636
637           mr.onstop = async () => {
638             setIsServerRecording(false);
639             const blob = new Blob(audioChunksRef.current, { type: "audio/webm" });
640             // send to server
641             await handleServerUpload(blob);
642             // stop the tracks to release mic
643             try {
644               stream.getTracks().forEach((t) => t.stop());
645             } catch {}
646           };
647
648           mr.onerror = (ev) => {
649             console.error("MediaRecorder error", ev);
650             setIsServerRecording(false);
651             try {
652               stream.getTracks().forEach((t) => t.stop());
653             } catch {}
654           };
655
656           mr.start();
657         } catch (e: any) {
658           console.error("startServerRecording:", e);
659           if (String(e).toLowerCase().includes("permission")) {
660             setMicPermissionDenied(true);
661           }
662           setIsServerRecording(false);
663         }
664       };
665
666       const stopServerRecording = () => {
667         try {
668           mediaRecorderRef.current?.stop();
669         } catch (e) {
670           console.warn("stopServerRecording:", e);
671           setIsServerRecording(false);
672         }
673       };
674
675       const handleServerUpload = async (blob: Blob) => {
```

```
        setIsTranscribing(true);
        setIsRecording(false);
        setInterimText("");
        baseBeforeRecordingRef.current = "";

        // Upload as form-data; expects backend /api/stt?lang=en|hi|kn
        const fd = new FormData();
        fd.append("file", blob, "voice.webm");

        try {
          const res = await fetch(`${API_BASE_URL}/api/stt?lang=${language}`, {
            method: "POST",
            body: fd,
          });
          if (!res.ok) {
            const txt = await res.text().catch(() => "");
            throw new Error(txt || `STT upload failed (${res.status})`);
          }
          const json = await res.json().catch(() => ({}));
          const transcript = (json && (json.transcript || json.text || json.result)) || "
          if (transcript && transcript.trim()) {
            // append transcript to input
            setInput((prev) => {
              const base = prev || "";
              const sep = base && !base.endsWith(" ") ? " " : "";
              return `${base}${sep}${transcript}`.trim() + " ";
            });
          } else {
            // if server returned nothing
            console.warn("Empty transcript from server", json);
          }
        } catch (err: any) {
          console.error("handleServerUpload error", err);
          alert("Failed to transcribe audio: " + (err?.message || "unknown"));
        } finally {
          setIsTranscribing(false);
          // ensure mediaRecorder cleaned up
          try {
            mediaRecorderRef.current = null;
            audioChunksRef.current = [];
          } catch {}
          textareaRef.current?.focus();
        }
      };

      // Start recognition (either browser or server) depending on strategy
      const startRecognition = async () => {
        const useServer = shouldUseServerStt(language, forceServerStt);

        // Snapshot base text so overlay can show it
        baseBeforeRecordingRef.current = input || "";

        if (useServer) {
          // server recording path
          await startServerRecording();
          return;
        }

        // browser SpeechRecognition path
        if (!supportsSpeechRecognition || !recognitionRef.current) {
          setMicPermissionDenied(true);
          return;
        }
```

```
739
740         try {
741           setInterimText("");
742           setIsRecording(true);
743           setMicPermissionDenied(false);
744           // set recognition language (map our language codes to locale)
745           let langToUse = "en-IN";
746           if (language === "hi") langToUse = "hi-IN";
747           if (language === "kn") langToUse = "kn-IN"; // fallback; still we'll prefer ser
748           recognitionRef.current.lang = langToUse;
749           try {
750             recognitionRef.current!.start();
751           } catch (e: any) {
752             console.warn("recognition.start() failed:", e);
753             if (String(e).toLowerCase().includes("not allowed") || String(e).toLowerCase(
754               setMicPermissionDenied(true);
755             }
756             setIsRecording(false);
757           }
758           textareaRef.current?.focus();
759         } catch (e) {
760           console.error("Failed starting recognition", e);
761           setIsRecording(false);
762         }
763       };
764
765       const stopRecognition = () => {
766         // stop both possible flows
767         try {
768           if (isServerRecording) {
769             stopServerRecording();
770           }
771         } catch {}
772         try {
773           recognitionRef.current?.stop();
774         } catch {}
775         setIsRecording(false);
776         setInterimText("");
777         baseBeforeRecordingRef.current = "";
778       };
779
780       const toggleMic = async () => {
781         // if currently transcribing, ignore toggles
782         if (isTranscribing) return;
783
784         // If server recording active -> stop
785         if (isServerRecording) {
786           stopServerRecording();
787           return;
788         }
789
790         // If browser recognition active -> stop
791         if (isRecording) {
792           stopRecognition();
793           return;
794         }
795
796         // start fresh
797         await startRecognition();
798       };
799
800       // ensure we stop recognition when unmounting
801       useEffect(() => {
```

```
802         return () => {
803           try {
804             if (recognitionRef.current) recognitionRef.current.abort?.();
805           } catch {}
806           try {
807             if (mediaRecorderRef.current) mediaRecorderRef.current.stop();
808           } catch {}
809         };
810       }, []);
811
812       // Small helper: mic SVG component
813       const MicSVG = ({ recording }: { recording: boolean }) => (
814         <svg
815           aria-hidden="true"
816           width="16"
817           height="16"
818           viewBox="0 0 24 24"
819           fill="none"
820           xmlns="http://www.w3.org/2000/svg"
821           className={`${recording ? "text-rose-500" : "text-slate-500"} w-4 h-4`}
822           role="img"
823           focusable="false"
824         >
825           <path d="M12 14a3 3 0 0 0 3-3V6a3 3 0 0 0-6 0v5a3 3 0 0 0 3 3z" stroke="current
826           <path d="M19 11v1a7 7 0 0 1-14 0v-1" stroke="currentColor" strokeWidth="1.5" st
827           <path d="M12 19v3" stroke="currentColor" strokeWidth="1.5" strokeLinecap="round
828         </svg>
829       );
830
831       // Helper label to show small status next to mic
832       const micStatusLabel = () => {
833         if (isTranscribing) return "Transcribing...";
834         if (isServerRecording) return "Recording (server)";
835         if (isRecording) return "Listening";
836         return "Mic";
837       };
838
839       return (
840         <div className={rootClass}>
841           {/* Accessible status live region for mic state */}
842           <div aria-live="polite" className="sr-only" role="status">
843             {isRecording || isServerRecording || isTranscribing ? "Microphone active" : "
844           </div>
845
846           {/* Header */}
847           <header
848             className={`border-b px-4 py-3 flex flex-col gap-1 sm:flex-row sm:items-cente
849           >
850             <div className="flex flex-col">
851               <h1 className="text-xl font-semibold tracking-tight">
852                 YantraBuddy
853               </h1>
854               <p className={`text-xs ${headerSubText}`}>
855                 Your source for everything on rock breakers, spare parts, and dealer insi
856               </p>
857             </div>
858
859             <div className="flex items-center gap-3 mt-1 sm:mt-0 text-[11px]">
860               <div className="flex items-center gap-2">
861                 {/* Language selector (client-only to avoid SSR mismatch) */}
862                 {isMounted && (
863                   <select
864                     value={language}
```

```
                  onChange={(e) => setLanguage(e.target.value as "en" | "hi" | "kn")}
                  className="text-xs px-2 py-1 rounded-full border bg-white dark:bg-sla
                  title="Language for speech"
                >
                  <option value="en">English</option>
                  <option value="hi">Hindi</option>
                  <option value="kn">Kannada</option>
                </select>
              )}

              {/* Force server STT toggle */}
              {isMounted && (
                <label className="text-xs inline-flex items-center gap-2 px-2 py-1 roun
                  <input
                    type="checkbox"
                    checked={forceServerStt}
                    onChange={(e) => setForceServerStt(e.target.checked)}
                    className="w-4 h-4"
                  />
                  <span className="text-[11px]">Server STT</span>
                </label>
              )}
            </div>

            <button
              type="button"
              onClick={toggleTheme}
              className="px-3 py-2 rounded-full border border-slate-500/40 bg-slate-900
            >
              {theme === "dark" ? "🌞 Light" : "🌙 Dark"}
            </button>
            <button
              type="button"
              onClick={handleClearChat}
              className="px-3 py-2 rounded-full border border-red-500/60 bg-red-500/10
            >
              🧹 Clear
            </button>
          </div>
        </header>

        {/* Main */}
        <main className="flex-1 flex flex-col max-w-4xl mx-auto w-full p-4">
          <section
            className={`${cardClass} w-full flex flex-col h-[82vh]`}
          >
            {/* Greeting + quick actions */}
            <div className={`p-4 border-b ${dividerBorder}`}>
              <p className="text-sm font-medium">
                {greeting}, welcome to YantraLive.
              </p>
              <p className="text-xs mt-1 text-slate-400">
                {hasMessages
                  ? "What else would you like to check? You can also try these question
                  : "How can I help you today? Here are some common questions:"}
              </p>

              <div className="mt-3 flex flex-wrap gap-2">
                {quickActions.map((q) => (
                  <button
                    key={q}
                    type="button"
                    onClick={() => handleQuickQuestion(q)}
```

```
                    className="text-xs px-3 py-1 rounded-full border border-sky-500/60
                  >
                    {q}
                  </button>
                ))}
              </div>
            </div>

            {/* Messages area */}
            <div
              ref={messagesContainerRef}
              className="flex-1 min-h-0 overflow-y-auto p-4 space-y-3"
            >
              {!hasMessages && (
                <div className="text-lg font-medium text-slate-300 text-center mt-20">
                  Ask anything about breakers, machine compatibility, spare
                  parts, or dealers.
                </div>
              )}

              {messages.map((m) => (
                <div
                  key={m.id}
                  className={`flex ${m.role === "user" ? "justify-end" : "justify-start
                >
                  <div className={`flex flex-col max-w-[80%] ${m.role === "user" ? "ite
                    <div className={`rounded-2xl px-3 py-2 text-sm whitespace-pre-wrap
                      {m.role === "assistant" ? (
                        <ReactMarkdown
                          remarkPlugins={[remarkGfm]}
                          components={{
                            table: ({ children }) => (
                              <div className="overflow-x-auto my-2">
                                <table className="text-xs border-collapse w-full">
                                  {children}
                                </table>
                              </div>
                            ),
                            th: (props) => (
                              <th
                                {...props}
                                className="border border-slate-600 px-2 py-1 bg-slate-9
                              />
                            ),
                            td: (props) => (
                              <td
                                {...props}
                                className="border border-slate-700 px-2 py-1 align-top"
                              />
                            ),
                            ul: (props) => (
                              <ul
                                {...props}
                                className="list-disc list-inside space-y-1"
                              />
                            ),
                            a: (props) => {
                              const href = String(props.href || "");
                              try {
                                const u = new URL(href, window.location.href);
                                if (u.pathname.startsWith("/brochures") || href.include
                                  return (
                                    <a
```

```
                                {...props}
                                href="#"
                                onClick={(e) => {
                                  e.preventDefault();
                                  openBrochureModal(href);
                                }}
                              >
                                {props.children}
                              </a>
                            );
                          }
                        } catch {
                          // fall back
                        }
                        return (
                          <a {...props} target="_blank" rel="noopener noreferrer"
                            {props.children}
                          </a>
                        );
                      },
                    }}
                  >
                    {boldParameterNames(preprocessAssistant(m.content))}
                  </ReactMarkdown>
                ) : (
                  m.content
                )}
              </div>

              <div className="mt-1 flex items-center justify-between w-full gap-2
                <span className={`text-[10px] ${timestampClass}`}>
                  {m.timestamp}
                </span>

                {m.role === "assistant" && (
                  <div className="flex items-center gap-1 text-[10px] text-slate-
                    <button
                      type="button"
                      onClick={() => handleCopy(m.content)}
                      className="px-1 py-[1px] rounded-full border border-slate-6
                      title="Copy reply"
                    >
                      📋
                    </button>
                    <button
                      type="button"
                      className="px-1 py-[1px] rounded-full border border-slate-6
                      title="Like"
                    >
                      👍
                    </button>
                    <button
                      type="button"
                      className="px-1 py-[1px] rounded-full border border-slate-6
                      title="Dislike"
                    >
                      👎
                    </button>
                  </div>
                )}
              </div>
            </div>
          </div>
```

```
1054                ))}
1055
1056                <div ref={messagesEndRef} />
1057                {loading && (
1058                  <div className="flex justify-start">
1059                    <div className="text-xs text-slate-400 px-3 py-2 bg-slate-800 rounded
1060                      Thinking based on the YantraLive datasets…
1061                    </div>
1062                  </div>
1063                )}
1064              </div>
1065
1066              {/* Input */}
1067              <form
1068                className={`border-t p-3 flex gap-2 rounded-b-2xl flex-shrink-0 ${divider
1069                onSubmit={(e) => {
1070                  e.preventDefault();
1071                  if (!loading) void handleSend();
1072                }}
1073              >
1074                {/* input area wrapper: relative so we can overlay interim text */}
1075                <div className="relative flex-1">
1076                  {/* Actual textarea - while recording we make text transparent and show
1077                  <textarea
1078                    ref={textareaRef}
1079                    className={`w-full text-sm rounded-xl border ${inputBorder} ${inputBg
1080                    rows={1}
1081                    placeholder="Type your question about breakers, machines, spare parts
1082                    value={input}
1083                    onChange={(e) => setInput(e.target.value)}
1084                    onKeyDown={handleKeyDown}
1085                    aria-label="Message input"
1086                    style={{ whiteSpace: "pre-wrap" }}
1087                  />
1088
1089                  {/* Overlay showing baseBeforeRecording + interim while recording */}
1090                  {(isRecording || isServerRecording) && (
1091                    <div
1092                      aria-hidden
1093                      className="absolute inset-0 pointer-events-none px-2 py-2 overflow-
1094                      style={{
1095                        whiteSpace: "pre-wrap",
1096                        wordBreak: "break-word",
1097                        display: "flex",
1098                        alignItems: "flex-start",
1099                      }}
1100                    >
1101                      <div className="w-full text-sm leading-[1.3]">
1102                        {/* show the base text snapshot (if any) */}
1103                        <span className="text-slate-200">{baseBeforeRecordingRef.current}
1104                        {/* show the interim in grey which updates live for browser STT;
1105                        {isRecording ? (
1106                          <span className="text-slate-400">{interimText}</span>
1107                        ) : (
1108                          <span className="text-slate-400"> ● Recording... </span>
1109                        )}
1110                        {/* show a caret indicator */}
1111                        <span className="inline-block w-1 h-4 align-middle bg-sky-400 ml-
1112                      </div>
1113                    </div>
1114                  )}
1115
1116                  {/* Microphone permission hint */}
```

```
1117                  {micPermissionDenied && (
1118                    <div className="absolute -bottom-5 left-0 text-xs text-rose-400">
1119                      Microphone access required — please allow the browser to use your m
1120                    </div>
1121                  )}
1122                </div>
1123
1124                {/* Mic button - render an initial server-safe placeholder; after mount r
1125                {!isMounted ? (
1126                  // server-rendered placeholder: disabled, fixed title — avoids hydratio
1127                  <button
1128                    type="button"
1129                    title="Speech recognition not supported"
1130                    aria-pressed={false}
1131                    className="px-3 py-2 rounded-xl border border-slate-300 hover:bg-slat
1132                    disabled
1133                  >
1134                    <span className="inline-flex items-center gap-2 text-sm">
1135                      <span aria-hidden>
1136                        {/* placeholder mic SVG (static) */}
1137                        <svg width="16" height="16" viewBox="0 0 24 24" fill="none" xmlns
1138                          <path d="M12 14a3 3 0 0 0 3-3V6a3 3 0 0 0-6 0v5a3 3 0 0 0 3 3z"
1139                          <path d="M19 11v1a7 7 0 0 1-14 0v-1" stroke="currentColor" stro
1140                          <path d="M12 19v3" stroke="currentColor" strokeWidth="1.5" stro
1141                        </svg>
1142                      </span>
1143                      <span className="text-xs">Mic</span>
1144                    </span>
1145                  </button>
1146                ) : (
1147                  // interactive mic button (client-side only after mount)
1148                  <div className="flex items-center gap-2">
1149                    <button
1150                      type="button"
1151                      onClick={toggleMic}
1152                      title={ supportsSpeechRecognition ? (isRecording || isServerRecordi
1153                      aria-pressed={isRecording || isServerRecording}
1154                      className={`px-3 py-2 rounded-xl border ${ (isRecording || isServer
1155                      disabled={isTranscribing}
1156                      aria-label={ (isRecording || isServerRecording) ? "Stop microphone"
1157                    >
1158                      <span className="inline-flex items-center gap-2 text-sm">
1159                        <span aria-hidden className="flex items-center">
1160                          <MicSVG recording={isRecording || isServerRecording} />
1161                        </span>
1162                        <span className="text-xs">{micStatusLabel()}</span>
1163                      </span>
1164                    </button>
1165
1166                    {/* show small transcribing loader if uploading/processing */}
1167                    {isTranscribing && (
1168                      <div className="text-xs px-2 py-1 rounded-md border bg-white dark:b
1169                        Transcribing...
1170                      </div>
1171                    )}
1172                  </div>
1173                )}
1174
1175                <button
1176                  type="submit"
1177                  disabled={loading || !input.trim()}
1178                  className="text-sm px-4 py-2 rounded-xl border border-sky-500 bg-sky-60
1179                >
```

```
                {loading ? "Sending..." : "Send"}
              </button>
            </form>
          </section>
        </main>

        {/* Brochure modal */}
        {brochureOpen && brochureUrl && (
          <div
            className="fixed inset-0 z-50 flex items-start justify-center p-6"
            aria-modal="true"
            role="dialog"
          >
            <div className="absolute inset-0 bg-black/60" onClick={closeBrochureModal}
            <div className="relative w-full max-w-4xl h-[85vh] bg-white dark:bg-slate-9
              <div className="flex items-center justify-between gap-4 p-3 border-b">
                <div className="flex items-center gap-3">
                  <span className="text-sm font-semibold">Brochure preview</span>
                  <span className="text-xs text-slate-500">{decodeURIComponent(brochure
                </div>
                <div className="flex items-center gap-2">
                  <a
                    href={brochureUrl}
                    target="_blank"
                    rel="noopener noreferrer"
                    className="text-xs px-3 py-1 rounded border border-slate-300 hover:
                  >
                    Open in new tab
                  </a>
                  <a
                    href={brochureUrl}
                    download
                    className="text-xs px-3 py-1 rounded border border-slate-300 hover:
                  >
                    Download
                  </a>
                  <button
                    onClick={closeBrochureModal}
                    className="text-sm px-3 py-1 rounded border border-red-300 text-red
                  >
                    Close
                  </button>
                </div>
              </div>

              <div className="w-full h-[calc(100%-56px)]">
                <iframe
                  src={brochureUrl}
                  title="Brochure preview"
                  className="w-full h-full"
                  sandbox="allow-same-origin allow-scripts allow-popups allow-forms"
                />
              </div>
            </div>
          </div>
        )}
      </div>
    );
  }
```